Secure and Practical Cold (and Hot) Staking*

Mario Larangeira

[1] Institute of Science Tokyo
rebello.m.f72a@m.isct.ac.jp
[2] Input Output Global (IOG)
mario.larangeira@iohk.io

Abstract. The stake delegation technique is what turns the general Proof of Stake (PoS) into a practical protocol for a large number of participants, ensuring the security of the distributed system, in what is known as Delegated PoS (DPoS). Karakostas et al. (SCN '20) formalized the delegation method paving the way for a whole industry of stake pools by proposing a formal definition for wallet as a universal composable (UC) functionality and introducing a corresponding protocol. On the other hand, a widely used technique named *hot/cold wallet* was formally studied by Das et al. (CCS '19 and '21), and Groth and Shoup (Eurocrypt '22) for different key derivation methods in the Proof of Work (PoW) setting, but not PoS. Briefly, while hot wallets are exposed to the risks of the network, the cold wallet is kept offline, thus more secure. However this may impair some capabilities given that the cold wallet is kept indefinitely offline. It is straightforward to observe that this "double wallet" design is not naturally portable to the setting where delegation is paramount, i.e., DPoS. This work identifies challenges for PoS Hot/Cold Wallet and proposes a secure and practical protocol.

Keywords: Delegated Proof of Stake · Stake Delegation · Hardware Wallet · Hot Wallet · Cold Wallet

1 Introduction

The notion of "addresses" transacting with each other was popularized by Bitcoin [21]. Each address is the hash value of the ECDSA verification key. In comparison with Proof of Work (PoW), typically, Proof of Stake (PoS) systems are more intensive in the use of cryptographic machinery. For example, whereas PoW relies mostly on signature scheme and hash function, generally PoS systems rely also in certificates (for delegation) and Verifiable Random Functions [20] for committee and leader elections [5, 9], let alone blockchain based platforms that allow smart contract applications that may rely in zero-knowledge protocols. Thus the system of addresses for a PoS system requires a cryptographically richer framework.

The first to propose such design was Karakostas *et al.* [14] with a universally composable wallet, following the Universally Composability (UC) Framework, specifically designed for PoS. This development is foundational for Delegated PoS and it has led to

*This work was supported by the JSPS KAKENHI under Grant JP21K11882.

the appearance of a stake pool economy with thousands of nodes underpinning the Cardano system [17]. Such system is a practical example of the wallet design framework in the wild with its security supported by thousands of stake pools [4].

On the other hand, Ethereum has chosen a rather different approach for its migration from PoW to PoS, *i.e.*, Ethereum 2.0. It relies on users allocating their stakes, and blocking them during a period of time [18], a significant change from [14], where the user can freely transact their tokens and the systems acknowledges the amount of stake distribution during leader election.

The DPoS setting is significantly different from the PoW one as [14] remarks. It seems the wallet security from one setting could not be trivially adapted to the other. An example is the hot/cold technique formalized in [8, 6] which, in a nutshell, allows the combination of two wallets: one with network access, hence the *hot wallet*, and the *cold wallet*, isolated and therefore secure. The crucial point is that the hot wallet can generate new cold wallet addresses in an "on demand" fashion while keeping the cold wallet offline via key derivation. How the delegation of the cold wallet, defined for PoW, where most of the funds are stored, works is unclear in DPoS.

This restriction refrains the cold wallet from spending, *or be drained from in the case of an attack*, the stored funds, because the wallet never comes online. However, given the ingenious design of the key derivation [8, 6], the permanently offline state *does not refrain* the cold wallet of receiving new funds. Furthermore, the cold wallet is often a *hardware wallet* given its typical extra security features. However it is not a mandatory requirement. That is the cold wallet can be a regular (offline) software client.

Such Hot/Cold approach for PoS based wallets, like those in [14], still remains unexplored. This gap in the research literature is surprising given the startling design differences between the wallets in [14] and [21]. In the light of the former, the addresses also encode a richer set of attributes, given is heavy use cryptographic machinery, and this capability does exist for regular ECDSA PoW addresses. Therefore it is far from clear whether the security analysis of [8, 6], which is based solely for PoW paradigm, preserves the security properties for DPoS setting, as shown in [14]. The very question

"How to perform the hot and cold wallet technique in DPoS?"

remains not fully answered. In order to better understand the question is necessary to look further into the DPoS staking protocol as it is currently performed.

Basics of Stake Delegation. We focus on the stake delegation style as proposed in [14] which is performed via certificates, in sharp contrast with procedure in [18] which locks funds. By the time a user publishes a *delegation certificate* in the ledger, the consensus protocol understands that the stake under control of that wallet is delegated to another verification key, *i.e.*, the stake pool, explicitly assigned in the certificate. The certificate also indicates the target of the rewards, which may not be the signer of the certificate. The set of stake pools perform the DPoS consensus protocol, which rewards every block generation according to the leader selection procedure. Note that, based on the published delegation certificates in the ledger, the stake distribution changes, therefore the probability of leader election, as common in PoS, changes accordingly. The elected leader, *i.e.*, stake pool, issues the new block.

DPoS Cold/Hot Staking Challenges. In the setting of a single wallet connected to the network, *a hot wallet*, it is straightforward to receive the rewards and perform delegation and *redelegation* as it is a matter of issuing a new certificate with a new signature. In the case of double wallet setting, forwarding the rewards would be straightforward since it is trivial to assign the rewards target, in the certificate, to the address of the cold wallet, *i.e.*, a cold address. On the other hand, it is unclear how the funds of the cold wallet can be staked since the signature in the certificate corresponds to the hot wallet. A similar issue occurs when the cold wallet needs to redelegate as it requires generating a signature and submitting it to the network; a major issue since the cold wallet is expected to be permanently disconnected.

The early cited limitations suggest a major rework on the delegation framework proposed in [14] in order to accommodate hot/cold wallet setting is necessary. It is also paramount that a novel design does not void the security guarantees offered by [14]. In summary, our early outlined research question *"How to perform the hot and cold wallet technique in DPoS?"* can be translated to the following questions

- **Cold stake delegation:** How the stake of the cold wallet is delegated, if the hot wallet signs the certificate?
- **Rewards payment:** How rewards are payed to the cold wallet?
- **Cold stake redelegation:** How redelegation is done? Does it require the cold wallet to be online?
- **Composable security:** Does a new hot/cold wallet construction design realizes the functionality in [14]?

Related Works. As briefly described, [14] proposed a thorough formalization of the addresses syntax and delegation method for PoS ledgers. In particular, its addresses system supports attributes embedded into the address strings. Moreover, it introduces two types of keys for (1) signing transactions and (2) performing stake delegation. In the PoW setting, the hot/cold wallet was first studied by Das *et al.* [8] in the setting of deterministic wallets [19], and their work focused on the key derivation of the ECDSA signature scheme which is widely employed for transactions in blockchain systems and, in particular, their stateful deterministic wallet design. Remarkably, [8] proposed changes in the Bitcoin Improvement Proposal 32 (BIP32) and its session derivation mechanism, *i.e.*, key derivation, in order to comply with their security modeling. In a follow up work, Das *et al.* [6] presented a formal security analysis of the BIP32 wallet standard as it is. This work build upon the security model introduced in [8].

While [6, 8] investigates the design of wallets, in a somewhat related work, Groth and Shoup [12] studied the two variations of the ECDSA scheme; the additive and multiplicative key derivation. Furthermore, their work also shed light to the role of "presignatures", a useful feature of the ECDSA in the threshold setting [2, 7, 16, 15]. The security of ECDSA based wallets is attracting interest given the recent finding on the fundamental limitations of the security of ECDSA as shown by Hartmann and Kiltz [13]. Despite of not mandatory, the cold wallet is often a hardware wallet, *i.e.*, piece of hardware with extra countermeasures against side-channels. Such class of wallets also received a formal treatment in [1] by Arapinis *et al.* in the UC Framework.

Finally a spendable cold wallet was, in fact, proposed with hardware aid [10], *i.e.*, optical channel, and is based in [8]. Unfortunately it is not designed for the DPoS setting. Nonetheless, the hardware aid could be adapted for the use in DPoS.

Our Contribution. In a nutshell, this work investigates the feasibility of relying on a similar design to the Hot/Cold Wallet in the DPoS setting. As a main requirement in this investigation, it is necessary to identify possible shortcomings in porting the Hot/Cold design from PoW to DPoS in current designs. Given this first step, we proceed in constructing a possible wallet/delegation framework to overcame such shortcomings.

Therefore, our contribution is three fold

- to identify limitations of the current Hot/Cold Wallet design [8] from the PoW setting to the DPoS [14], *i.e.*, perform stake delegation with a hot/cold wallet in PoS;
- to introduce a novel wallet design, *i.e.*, the protocol π_{HC} and the functionality $\mathcal{F}_{\mathrm{HC}}$, in order to support stake delegation, at the same time is offers enhanced security guarantees based on cold storage;
- to present a formal and thorough security analysis in the UC Framework, and prove that our PoS Hot/Cold Wallet Protocol π_{HC} UC realizes the UC Functionality $\mathcal{F}_{\mathrm{HC}}$.

Next Sections. We start by reviewing the basic designs for Hot/Cold Wallet by [8] and the delegation framework we focus, *i.e.*, [14], in Section 2. Section 3 presents a discussion on the shortcomings of the currently attempt to reuse previous work, *i.e.*, Stateful Hot/Cold Wallet, into the delegation framework based on certificates. Next, we fill the identified gaps from the early discussion in the previous section by presenting a new construction in Section 4, and introduce its security analysis in Section 5. Finally we conclude this work by presenting final remarks in Section 6 along with future work.

2 Preliminaries

In the next definitions, we consider negl to be a negligible function with respect to the security parameter κ. Moreover, in general we also assume that the adversary is Probabilistic Polynomial Time (PPT) also with respect to κ.

2.1 Digital Signature Scheme

We start by reviewing a very basic definition.

Definition 1 (Digital Signature). *A secure digital signature scheme, as in [3, 11], is triple of algorithms* $\Sigma = \langle \mathsf{Gen}, \mathsf{Verify}, \mathsf{Sign} \rangle$. Σ *is said to be resistant to Existential Unforgeable under Adaptive Chosen Message Attacks (EUF-CMA) if it presents the following properties with respect to the security parameter* κ*:*

Completeness: *For any message m, it holds* $\Pr[(\mathsf{vk},\mathsf{sk}) \leftarrow \mathsf{Gen}(1^\kappa), \sigma \leftarrow \mathsf{Sign}(\mathsf{sk},m) : 0 \leftarrow \mathsf{Verify}(m,\sigma,\mathsf{vk})] \leq \mathsf{negl}$*;*

Non-repudiation: *For any message m, the probability that two independent executions of* $\mathsf{Verify}(m,\sigma,\mathsf{sk})$*, such that* $(\mathsf{vk},\mathsf{sk}) \leftarrow \mathsf{Gen}(1^\kappa)$*, output two different outcomes is smaller than* negl*;*

Unforgeability: *For any PPT algorithm* $\mathcal{A}_{forger}$*, which can query the signature oracle* $\mathsf{Sign}(\mathsf{sk},\cdot)$ *for signatures on a polynomial number of messages* m_i*, it holds:* $\Pr[(\mathsf{vk},\mathsf{sk}) \leftarrow \mathsf{Gen}(1^\kappa) : (m,\sigma) \leftarrow \mathcal{A}_{forger}^{\mathsf{Sign}(\mathsf{sk},\cdot)} \wedge m \neq m_i] < \mathsf{negl}$*, where all the probabilities are computed over the random coins of the adversary and the signature algorithms.*

2.2 The Stake Delegation in PoS

One of the key points in the of the core wallet design π_{Wallet} of [14], is that the address system relies on two pairs of keys: one for spending transactions while the other is for the delegation. That is, $(\mathsf{sk}^{pay}, \mathsf{pk}^{pay})$ for payment, *i.e.*, signing regular transactions, and $(\mathsf{sk}^{stk}, \mathsf{pk}^{stk})$ for delegation, *i.e.*, signing of the delegation certificate. A wallet within this design, generates addresses such that could be matched with the delegation certificate, therefore the pk^{stk} can be directly matched with the signed certificate, allowing the consensus protocol to associate the stake of an address to that key: a crucial step in stake delegation. Needless to say, it relates directly to the security of the consensus protocol as it defines the stake distribution among all the stake pools, and they carry the consensus protocol.

Attributes and Addresses. Along with both pairs of keys, for staking and paying, [14] relies on collision resistance hash function, and introduces procedures to generate addresses α and assignment attributes to them: $\mathsf{GenAddr}$ and $\mathsf{HKeyGen}$.

For completeness, now we review these properties, including the standard collision resistance for hash function as given by [14].

Definition 2 (Collision resistance). *A function H is collision resistant if, given $h \leftarrow \{0,1\}^{\ell}$, it is computationally infeasible for a PPT algorithm to find a value x such that $h = \mathcal{H}(x)$ for some size ℓ.*

We now review the properties specific for the framework in [14] with respect to the address generation. Intuitively, they are designed to capture (and avoid) adversarial attempts to derivate malicious address strings from honest ones.

Definition 3 (Address collision resistance). *Analogously to hash functions,* $\mathsf{GenAddr}$ *is collision resistant when it is infeasible to produce two different attribute lists $l_i = (\delta_1^i, \ldots, \delta_g^i)$ for $i \in \{1,2\}$,* i.e.*, they differ in at least one attribute like $\exists j \in [1,g]$: $\delta_j^1 \neq \delta_j^2$, such that $\mathsf{GenAddr}(l_1) = \mathsf{GenAddr}(l_2)$, after running $\mathsf{GenAddr}(\cdot)$ a polynomial number of times.*

Definition 4 (Hierarchical Key Generation). *For a key generation function $\mathsf{HKeyGen}(\cdot)$ and signature scheme $\Sigma = \langle \mathsf{Gen}, \mathsf{Verify}, \mathsf{Sign} \rangle$, $\mathsf{HKeyGen}(\cdot)$ is hierarchical for Σ if, for all w, the key distribution produced by $\mathsf{HKeyGen}(w)$ is computationally indistinguishable from the key distribution produced by Gen.*

The address generation depends on the key and meta information attributes [14]. For completeness, we recall the following definition.

Definition 5 (Non-malleable attribute address generation). *Let $\mathcal{L}$ be a distribution of attribute lists and $l \leftarrow \text{DOM}(\mathcal{L})$ an attribute list, such that $\text{DOM}(\mathcal{L}) = \Delta_1 \times \ldots \times \Delta_g$. Let the first attributes of l $\delta_1, \ldots, \delta_i$ relate to a property over which we define non-malleability. Given an address α, it is infeasible for an adversary $\mathcal{A}$ to produce valid forgeries,* i.e.*, acceptable addresses with the same payment key as α, without access*

to α*'s private attributes,* even with access to the address's metadata*,* i.e.*, its semi-public attributes. Concretely, with* Addrs *the list of addresses queried by* $\mathcal{A}$ *to the oracle* $\mathsf{GenAddr}^d(\cdot)$*, it holds that*

$$\Pr\left[\begin{array}{l} l=(\delta_1,\ldots,\delta_g),\ \alpha \leftarrow \mathsf{GenAddr}(l), \\ \mathcal{A}^{\mathsf{GenAddr}(\cdot),\mathsf{GenMeta}(\cdot)}(\delta_i,\ldots,\delta_g) \rightarrow \\ \qquad\qquad (\alpha',\delta'_i,\ldots,\delta'_g) \end{array} : \begin{array}{r} (\mathsf{GenAddr}(\delta'_i,\ldots,\delta'_g)=\alpha')\wedge\ (\alpha'\neq\alpha)\wedge \\ (\alpha'\notin \mathsf{Addrs}) \end{array}\right] \leq \mathsf{negl}$$

for probabilities over the random coins of GenAddr *and the PPT adversary* $\mathcal{A}$*, and choices of* l*.*

The access to the oracles $\mathsf{GenAddr}^d$ and $\mathsf{GenMeta}^d$ captures the scenario where the adversary has access to a source of well formatted addresses and metadata which may contain useful information.

PoS Wallet Interface and Stake Pool. As mentioned earlier, the PoS design requires more cryptography machinery than the PoW approach, therefore the wallet π_{Wallet}, as devised in [14], introduces separate actions for *payment*, *delegation* (by putting forward PAY and STAKE interfaces), and also *stake pool registration*. For the latter, their design introduces space for metadata *meta* and makes use of a regular payment transaction whose syntax we introduce next for completeness.

Payment: It is the transfer of Θ assets from a sender's to receiver's addresses α_s and α_r via transaction $tx=(\Theta,\alpha_s,\alpha_r,meta)$, with metadata *meta*, *i.e.*, fee amount. Next it accesses the PAY interface of π_{Wallet}, retrieves the signed (by the payment secret key sk^{pay}) tx, and publishes it on the ledger.

Stake pool registration: Every pool is identified by a registered staking key. That is, the pool operator relies on π_{Wallet} to produce a new staking key pair $(\mathsf{sk}^{stk}_{pool}, \mathsf{pk}^{stk}_{pool})$, which the operator receives pk^{stk}_{pool}. It, then, creates a registration certificate $\mathcal{C}_{reg} = (\mathsf{pk}^{stk}_{pool}, meta)$, where *meta* is the pool's metadata, *e.g.*, the name of the pool's operator. The certificate is signed by π_{Wallet} with sk^{stk}_{pool}, and the signature is returned. The certificate $\mathcal{C}_{reg}$ and the received signature are published on the ledger (in a payment transaction), thus registering pk^{stk}_{pool} on behalf of the pool.

Delegation/Redelegation: The stake delegation enables a wallet to assign a stake pool to perform staking on its behalf. It is based on delegation certificates like $\mathcal{C}_{del} = (\mathsf{pk}^{stk}_{user}, \langle \mathsf{pk}^{stk}_{pool}, meta\rangle)$. Namely, pk^{stk}_{user} is the staking key to which the certificate applies, *i.e.*, the key which controls the stake of an address, pk^{stk}_{pool} is the delegate's key, *i.e.*, the registered key of a stake pool, and *meta* is the certificate's metadata. The wallet gives $\mathcal{C}_{del}$ to π_{Wallet} to be signed with sk^{stk}_{user}, then it publishes $\mathcal{C}_{del}$ and the signature via a payment transaction (as in the stake pool registration).

Remark 1. When a staking key issues a delegation certificate, *all* addresses which are associated with it are re-delegated accordingly. For instance, if a pointer address [3] points to a key pk^{stk}_{user}, then the latest certificate issued by this key takes effect.

[3] In the jargon of [14] it is an type of address that does not really have a cryptography key of its own, instead in its attributes it "points" to another public key. A stake pool re-delegates via a

2.3 The Stateful Hot/Cold Wallet

For completeness, we review [8]. The following definition is important because we rely on it in order to keep the key in the cold storage synchronized with the hot wallet via key derivation.

Definition 6 (Stateful Hot/Cold Wallet Scheme [8]). *For a security parameter* κ*, a Stateful Hot/Cold Wallet is defined by the algorithms* $\Sigma_{HC} = (\mathsf{MGen}, \mathsf{SKDer}, \mathsf{PKDer}, \mathsf{WSign}, \mathsf{WVer})$ *such that*

- $\mathsf{MGen}(1^\kappa, \rho) \to (\mathtt{st}_0, \mathtt{msk}, \mathtt{mpk})$*:The probabilistic master key generation algorithm* MGen *takes as input the security parameter* 1^κ *and randomness* ρ*, then outputs an initial state* $\mathtt{st}_0$ *that is given to both hot and cold wallets, a master public key* mpk *that is given to the hot wallet, and a master secret key* msk *that is given to the cold wallet;*
- $\mathsf{SKDer}(\mathtt{msk}, \mathtt{st}, \mathsf{id}) \to (\mathsf{sk}_{\mathsf{id}}, \mathtt{st}')$*: The deterministic secret key derivation algorithm* SKDer *is run by the cold wallet to derive a session secret key. It takes as input the master secret key* msk*, the state* st*, identifier* id*, and outputs a session secret key* $\mathsf{sk}_{\mathsf{id}}$ *and the new state* $\mathtt{st}'$*;*
- $\mathsf{PKDer}(\mathtt{mpk}, \mathtt{st}, \mathsf{id}) \to (\mathsf{pk}_{\mathsf{id}}, \mathtt{st}')$*: The deterministic public key derivation algorithm* PKDer *is run by the hot wallet to derive a session public key. It takes the master public key* mpk*, the state* st *and an identifier* id*, and outputs a session public key* $\mathsf{pk}_{\mathsf{id}}$ *and the new state* $\mathtt{st}'$*;*
- $\mathsf{WSign}(\mathtt{m}, \mathsf{sk}) \to \sigma$*: The probabilistic signing algorithm* WSign *is executed by the cold wallet. It takes as input a message* m *and a session secret key* sk *and outputs a signature* σ*;*
- $\mathsf{WVer}(\mathtt{m}, \sigma, \mathsf{pk}) \to \{0,1\}$: *The deterministic verification algorithm* WVer *is executed by any party that wants to verify the validity of a signature. It takes as input a session public key* pk*, a message* m *and a signature* σ*, and outputs* 0 *or* 1*.*

We stretch the security definition for signature, *i.e.*, Definition 1, to Definition 6 by saying that Σ_{HC} is EUF-CMA if it shows similar properties.

Remark 2. The identifier id is used to construct a new public key $\mathsf{pk}_{\mathsf{id}}$ for an arbitrary choice of id by allowing a common secret string *ch*, named "chaincode", between the wallets. As argued in [8, 19], for the ECDSA signature scheme, in order to derive a new key $\mathsf{pk}_{\mathsf{id}}$ while preserving the unlinkability and unforgeability security properties, it is necessary to compute $w \leftarrow \mathcal{H}(\mathsf{id}, ch)$, $\mathsf{pk}_{\mathsf{id}} \leftarrow \mathtt{mpk} + w \cdot G$ and $\mathsf{sk}_{\mathsf{id}} \leftarrow \mathtt{msk} + w$, for a hash function $\mathcal{H}$ and the ECDSA key pair $\mathtt{msk} = x$ and $\mathtt{mpk} = x \cdot G$. We omit the value *ch* later for readability.

Correctness of a Stateful Hot/Cold Wallet Scheme. The correctness requirement guarantees that if the cold/hot wallets derive session key pairs on the same set of identifiers $\mathsf{id}_1, \ldots, \mathsf{id}_n$ and in the same order (departing from the initial state $\mathtt{st}_0$), then any signature created by the cold wallet using one of the session secret keys $\mathsf{sk}_{\mathsf{id}_i}$ are accepted when the verification algorithm is executed with the corresponding session public key $\mathsf{pk}_{\mathsf{id}_i}$. In other words, all the derived session secret and public keys should match.

lightweight certificate, *i.e.*, a certificate not published via a transaction but included in the block's headers.

Remark 3. In our protocol we assume the existence of a deterministic function $I : \mathbb{N} \rightarrow \{0,1\}^*$ to generate unique identifiers $\mathsf{id}_1, \ldots, \mathsf{id}_n$, to identify the key sessions between the wallets. Concretely, given the sequential index $i \in \mathbb{N}$, both wallets can be kept synchronized by the correct generation of id_i values, *i.e.*, executing $I(i) = \mathsf{id}_i$. For an arbitrary id^* value, the cold wallet can check the equality $I(i) = \mathsf{id}_i = \mathsf{id}^*$ for sequential values i and confirm the correct index for the session key id^*. This confirmation procedure allow the pair of wallets to be kept synchronized regarding the session keys.

Security of the Stateful Hot/Cold Wallet Scheme. In Das *et al.* [8], the Stateful Hot/Cold Wallet Scheme should satisfy two security properties: *Unlinkability* and *Unforgeability*. Intuitively, the former protects the privacy of the transaction receiver and captures the guarantee that it should be infeasible to link transactions sending funds to different session public keys that were derived from the same master public key.

It is required that an adversary that is given the master public key cannot distinguish between session public keys derived from that master public key, and session public keys that are generated from a fresh (*i.e.*, independently and randomly chosen) master public key. As highlighted in [8], such security property cannot be achieved for keys which the adversary knows the state used to derive them (the adversary can learn such state if there is a breach in the hot wallet, for example).

The Unforgeability property captures the feature that once funds are transferred to the cold wallet, they remain secure if: (1) the hot wallet is breached; and (2) the adversary observes transactions signed by the cold wallet. Even such adversary cannot generate new valid transactions spending cold wallet funds.

3 PoS Hot/Cold Limitations

The current design, outlined in Section 2, presented features that can be used to perform delegation in the hot/cold wallet setting. On the other hand, some features are missing and others are not adequate. We thoroughly discuss each one of them next.

- **Cold stake delegation:** As pointed in Section 2.2, the delegation is crucially dependent of the publication of the delegation certificate $\mathcal{C}_{del} = (\mathsf{pk}^{stk}_{user}, \langle \mathsf{pk}^{stk}_{pool}, meta \rangle)$ and σ_{del} generated by sk^{stk}_{user}. The construction to be presented has to set the *cold wallet* key pairs, *i.e.*, for staking and payment, and manage them accordingly;
- **Rewards payment:** The delegation certificate, *i.e.*, $\mathcal{C}_{del} = (\mathsf{pk}^{stk}_{user}, \langle \mathsf{pk}^{stk}_{pool}, meta \rangle)$, and *meta* can contain the (see next *Cold staking redelegation*) cold wallet address to where the rewards can be moved. The pool operator uses it to deposit the rewards;
- **Cold stake redelegation:** Every new delegation, *i.e.*, for redelegation, a new address can be issued under the control of the cold wallet, *i.e.*, controlled by sk^{stk}_{user};
- **Composable security:** The protocol π_{Wallet} from [14] seems to be incompatible to the Stateful Hot/Wold Wallet, *i.e.*, Definition 6, in the current form, which suggests a redesign of π_{Wallet} in order to realize the security definition of [14];
- **Hot wallet funds:** Given that the delegation is performed by certificate publication in the ledger via a regular (payment) transaction, the hot wallet, *i.e.*, sk^{pay}, must to have some funds in order to pay (and also stake) regular fees for publication.

In summary, given the earlier discussion, it is not clear that we can port the current stateful hot/cold wallet design into DPoS.

Next, we introduce the Hot/Cold Wallet protocol which works in pair with a cold storage. The protocol π_{HC}, to be presented next, exploits the structure of double key address structure proposed in [14] to put forth a cold staking delegation technique.

4 PoS Stateful Hot/Cold Wallet Protocol π_{HC}

The main motivation of our next definition is to fill the gaps discussed in Section 3. In order words, next we introduce our protocol π_{HC}. Before we detail our protocol, it is convenient to outline the main intuition and the basic procedures that we rely for the issuing of keys and addresses.

4.1 Intuition: Four Pairs of Keys

We rely on four pairs of keys: $(\mathsf{sk}^{stk}_{hot}, \mathsf{pk}^{stk}_{hot})$, $(\mathsf{sk}^{pay}_{hot}, \mathsf{pk}^{pay}_{hot})$, and $(\mathsf{sk}^{stk}_{cold}, \mathsf{pk}^{stk}_{cold})$ are key pairs from regular (ECDSA) signature in the sense of Definition 1, whereas the key pair $(\mathtt{msk}^{pay}_{cold}, \mathtt{mpk}^{pay}_{cold})$ is a Stateful Hot/Cold Wallet key pair in the sense of Definition 6. Thus the latter is generated as outlined in Section 2.3:

- ECDSA key pair $\mathtt{msk}^{pay}_{cold} = x$ and $\mathtt{mpk}^{pay}_{cold} = x \cdot G$, for uniformly random x;
- Derivation information generation: $w \leftarrow \mathcal{H}(\mathsf{id}, ch)$, for a "chaincode" ch and index id;
- Keys derivation: $\mathsf{pk}^{pay}_{c,\mathsf{id}} \leftarrow \mathtt{mpk}^{pay}_{cold} + w \cdot G$ and $\mathsf{sk}^{pay}_{c,\mathsf{id}} \leftarrow \mathtt{msk}^{pay}_{cold} + w$, for a group generator G;
- Existence of a deterministic function $I : \mathbb{N} \to \{0,1\}^*$, such that the cold wallet can check the equality $I(i) = \mathsf{id}_i = \mathsf{id}^*$ for sequential values i.

4.2 Basic Procedures: Key and Address Generation

As expected, our protocol is based on the one from [14]. The main difference is that here we have to capture two wallets, namely, the hot and the cold one. Hence, we extend the protocol from [14], with the option of distinguishing the option between the wallets.

The main idea is that a user $\mathcal{U}$ has access to π_{HC} for the basic procedures for a wallet: initialize, pay (signing transaction), verify transaction, staking, generating address, recovery, etc.

As a preparation to introduce our full construction for π_{HC}, we present the concrete parameters for the protocol, which were originally proposed in [14].

- GenAddr (Algorithm 1): It is the malleable address generation function [4] which generates the address strings. Concretely it is given by Algorithm 1, and it supports three types of addresses [5]: "base", "pointer" and "exile";
  It is not hard to verify the following lemma.

Lemma 1. GenAddr *is collision resistant if* $\mathcal{H}$ *is collision resistant.*

[4] In [14], this function is referred as Ex Post Malleability type of address, and it is one of various degrees of malleability supported by their framework. Later the functionality receives an equivalente malleability predicative.

[5] The extra types of addresses are to support different entities like regular user, *i.e.*, "base", stake pools, *i.e.*, "pointer", not participants of PoS consensus, *i.e.*, "exile".

function $\mathsf{GenAddr}(l_\alpha)$
$(aux, st, \mathsf{sk}^{pay}, \mathsf{pk}^{pay}, wallet) = \mathsf{parse}(l_\alpha)$
switch("aux") **do**
case("$base$") : $\beta = \mathcal{H}(st)$
case("$pointer$") : $\beta = \mathsf{getPointer}(st)$
case("$exile$") : $\beta = st$
$\alpha = \mathcal{H}(\mathsf{pk}^{pay}_{wallet} || \mathsf{pk}^{stk}_{wallet} || \mathcal{H}(\mathsf{pk}^{pay}_{wallet})) || \beta || \mathcal{H}(\mathsf{pk}^{pay}_{wallet})$
return α

Algorithm 1: ($\mathsf{GenAddr}$). The malleable address generation function, parameterized by $\mathcal{H}(\cdot)$ and the Stateful Hot/Cold Wallet, *i.e.*, Definition 6. The input is a tuple l_α of the auxiliary information *aux* and attributes for address α.

– $\mathsf{HKeyGen}$ (Algorithm 2): It is the procedure which is used in combination of the $\mathsf{GenAddr}$. For every new address generation, the generation relies on a derivation of a signature public/secret key pair. In our construction, we use this procedure in addition to the Stateful Hot/Cold Wallet, *i.e.*, Definition 6.

function $\mathsf{HKeyGen}(\kappa, r, \langle label, wallet, index\rangle, (mpk, st))$
if label="pay" $\wedge$ wallet ="cold" **do**
return
$\mathsf{PKDer}(mpk, st, index) = (\mathsf{pk}^{pay}_{c,index}, st')$
else
$v \leftarrow PRF(r||label||wallet||index)$
$\rho \leftarrow PRNG(v)$
return$\mathsf{Gen}(1^\kappa, \rho) = (\mathsf{sk}^{label}_{wallet}, \mathsf{pk}^{label}_{wallet})$

Algorithm 2: ($\mathsf{HKeyGen}$). Hierarchical Key Generation, relies on a Pseudorandom Function (PRF), and Pseudorandom Number Generation (PRG). It is designed for $label \in \{$"pay","stk"$\}$ and $wallet \in \{$"hot","$cold$"$\}$.

– $\mathsf{RTagGen}$: It is the Recovery *Tag* Generation algorithm used in the recovery procedure of the wallet. The tags are generated by hashing the input with the collision resistant hash function $\mathcal{H}$.

4.3 Cold Storage and the π_{HC} Wallet

The cold storage and the wallet π_{HC} work in pair. While the former keeps the cold key $\mathtt{msk}_{cold}$, and therefore sk^{pay}_{cold}, the latter keeps remaining keys, and can perform the hierarchical key generation for address issuing. The initial cold key state, along with the $\mathtt{mpk}_{cold}$ and session key id, is passed to π_{HC}.

Concretely, $\mathcal{U}$ executes $\mathsf{MGen}(1^\kappa, \rho) \rightarrow (\mathtt{st}_0, \mathtt{msk}^{pay}_{cold}, \mathtt{mpk}^{pay}_{cold})$ for a randomness ρ, pick a random id, and send $(\textsc{Init}, sid, \mathtt{st}_0, \mathtt{mpk}^{pay}_{cold}, \mathsf{id})$ to π_{HC} as it can be seen in the *Initialization Interface* of π_{HC} next. The address generation starts by picking "child attributes" by the jargon of [14].

Protocol π_{HC}

Initialization:

Upon receiving $(\text{INIT}, sid, \mathtt{st}_0, \mathtt{mpk}^{pay}_{cold}, \mathsf{id})$ from $\mathcal{U}$, set $r \xleftarrow{\$} \{0,1\}^\kappa$ and return (INITOK, sid).

Address Generation:

-Upon receiving the message $(\text{GENERATEADDRESS}, sid, aux, wallet)$ from $\mathcal{U}$, compute the index and "child" attributes as follows: i) pick $i \leftarrow I$; ii) if wallet="cold", set $store = (\mathtt{mpk}^{pay}_{cold}, \mathtt{st}_0)$ and $index = \mathsf{id}$, else set $store = \perp$ and $index = i$ iii) set $(\mathsf{pk}^{chd,pay}_{wallet}, \mathsf{sk}^{chd,pay}_{wallet}) = \mathsf{HKeyGen}(\kappa, r, \langle \text{"pay"}, wallet, index\rangle, store)$; iv) set $wrt = \mathsf{RTagGen}(\mathsf{pk}^{chd,pay}_{wallet})$.

-If $aux =$ "base", set $(\mathsf{pk}^{chd,stk}, \mathsf{sk}^{chd,stk}) = \mathsf{HKeyGen}(\kappa, r, \langle \text{"stk"}, wallet, i\rangle, \perp)$; else, if $aux = (\text{"pointer"}, \mathsf{pk}^{stk})$,find $(\mathsf{pk}^{chd,stk}, \mathsf{sk}^{chd,stk}) \in K : \mathsf{pk}^{stk} = \mathsf{pk}^{chd,stk}$; else if $aux =$ "exile", set $(\mathsf{pk}^{chd,stk}, \mathsf{sk}^{chd,stk}) = (\perp, \perp)$.

-Then insert $l_\alpha = \langle \mathsf{pk}^{chd,stk}, wrt, aux, \mathsf{pk}^{chd,pay}_{wallet}, \mathsf{sk}^{chd,pay}_{wallet}, \mathsf{sk}^{chd,stk}_{wallet}, wallet\rangle$, generate new address α as $\alpha = \mathsf{GenAddr}(\langle aux, \mathsf{pk}^{stk}_c, \mathsf{sk}^{pay}_c, wrt\rangle)$, and insert the tuple $\langle \alpha, (\mathsf{pk}^{pay}_c, \mathsf{sk}^{pay}_c)\rangle$ to P^{wallet}_{key}.

-Return $(\text{ADDRESS}, sid, \alpha)$ to $\mathcal{U}$. If $aux =$ "base" also insert $(\mathsf{pk}^{chd,stk}_{wallet}, \mathsf{sk}^{chd,stk}_{wallet})$ to S^{wallet}_{key} and send the message $(\text{STAKINGKEY}, sid, \mathsf{pk}^{chd,stk}_{wallet})$ to $\mathcal{U}$.

Wallet Recovery: *// For both cold and hot*

Upon receiving the message $(\text{RECOVERWALLET}, sid, wallet, i_{max})$ from $\mathcal{U}$, $\forall i \in [0, i_{max}]$ set $(\mathsf{pk}^{pay}_i, \mathsf{sk}^{pay}_i) = \mathsf{HKeyGen}(\kappa, r, \langle \text{"}pay\text{"}, wallet, i\rangle, store)$, $store = \{(\mathtt{mpk}^{pay}_{cold}, \mathtt{st}_0), \perp\}$, return the message $(\text{TAG}, sid, \mathsf{RTagGen}(\mathsf{pk}^{pay}_i))$ to $\mathcal{U}$.

Address Recovery: *// For both cold and hot*

Upon receiving the message $(\text{RECOVERADDR}, sid, wallet, \alpha, i_{max})$ from $\mathcal{U}$, parse the address's attributes $(\mathsf{pk}^{stk}, wrt, aux) = parsePubAttrs(\alpha)$. If exists $i \in I : i < i_{max}$, where $(\mathsf{pk}^{pay}_i, \mathsf{sk}^{pay}_i) = \mathsf{HKeyGen}(\kappa, r, \langle \text{"pay"}, wallet, i\rangle, store)$, for $store = \{(\mathtt{mpk}^{pay}_{cold}, \mathtt{st}_0), \perp\}$ and $\mathsf{RTagGen}(\mathsf{pk}^{pay}_i) = wrt$, and then return the message $(\text{RECOVEREDADDR}, sid, \alpha)$.

Issue Hot Transaction: *// Only hot payment*

Upon receiving the message $(\text{PAY}, sid, tx = (\Theta, \alpha_s, \alpha_r, m))$ from $\mathcal{U}$, find $\langle \alpha_s, (\mathsf{pk}^{pay}, \mathsf{sk}^{pay})\rangle \in P^{hot}_{key}$ and return the $(\text{TRANSACTION}, sid, tx, \mathsf{Sign}(\mathsf{sk}^{pay}, tx))$.

Issue Cold Transaction: *There is no interface because sk^{pay}_{cold} is disconnected in cold storage.*

Verify Transaction: *// For both cold and hot*

Upon receiving $(\text{VERIFYPAY}, sid, tx, \sigma)$ from $\mathcal{U}$, with $tx = (\Theta, \alpha_s, \alpha_r, meta)$ for some metadata $meta$, find an entry $\langle \alpha_s, (\mathsf{pk}^{pay}, \mathsf{sk}^{pay})\rangle$ in P^{wallet}_{key}, for either $wallet \in \{\text{"}hot\text{"}, \text{"}cold\text{"}\}$, and return $(\text{VERIFIEDPAY}, sid, tx, \sigma, \mathsf{WVer}(tx, \sigma, \mathsf{pk}^{pay}))$ to $\mathcal{U}$.

Issue Staking: *// Staking secret key is accessible*

Upon receiving $(\text{STAKE}, sid, stx, wallet)$ from $\mathcal{U}$ such that $stx = (\mathsf{pk}^{stk}, meta)$, for some metadata $meta$ and find $(\mathsf{pk}^{stk}, \mathsf{sk}^{stk}) \in S^{wallet}_{key}$ for either $wallet \in \{hot, cold\}$, then return $(\text{STAKED}, sid, stx, \mathsf{Sign}(\mathsf{sk}^{stk}, stx))$.

Verify Staking: *// For both cold and hot*
Upon receiving $(\text{VERIFYSTAKE}, sid, stx, \sigma)$ from party $\mathcal{U}$, where $stx = (\mathsf{pk}, meta)$ for some metadata $meta$, then find $(\mathsf{pk}^{stk}, \mathsf{sk}^{stk}) \in S_{key}^{wallet}$, for $wallet \in \{hot, cold\}$, return $(\text{VERIFIEDSTAKE}, sid, stx, \sigma, \mathsf{WVer}(stx, \sigma, \mathsf{sk}^{stk}))$.

Remark 4. The trick is that the cold storage only stores the cold secret key for payment. When sk_{cold}^{pay} is stored, it is in fact storing the new state $\mathtt{st}'$ to be used in the new key derivation. In order to perform the online redelegation, we only need the cold secret key for *staking*. That is, we do not need the key for signing a payment transaction (from the cold wallet). Thus the double key structure, for payment and staking, comes handy. Thus we can freely do (re)delegation even with the cold wallet offline permanently.

5 Security Analysis

In [14], the main security definition is the wallet functionality $\mathcal{F}_{\text{Wallet}}^{M}$, which receives, as a parameter, the malleability predicative[6] M, in order to propose a flexible definition for different degrees of address malleability. That is, how much the adversary can reuse the same address with different staking keys; the so called malleability issue identified in [14]. Unfortunately, $\mathcal{F}_{\text{Wallet}}^{M}$ does not support hot/cold design, therefore from now we proposed a redesign of $\mathcal{F}_{\text{Wallet}}^{M}$ into $\mathcal{F}_{\text{HC}}^{M}$ in order to analyze the previously presented wallet protocol π_{HC}.

5.1 The Functionality $\mathcal{F}_{\text{HC}}$

Before presenting our functionality $\mathcal{F}_{\text{HC}}$, for completeness, we recall the malleability predicate M which is a parameter for $\mathcal{F}_{\text{HC}}$.

function $M^{\mathbb{L},\mathbb{T},\mathcal{U}}(aux, \alpha)$
  switch("aux")
    case("issue") : **return** 1
    case("verify" *OR* "recover") : $d = \text{parsePubAttrs}(\alpha)$
      for $\delta \in d$ **do**
        if $\forall \alpha' \in L_{\mathcal{U}} \in , d' = \text{parsePubAttrs}(\alpha')$: $\delta \notin d'$ **then return** 0
        if $\forall (\Theta, \alpha_s, \alpha_r, m) \in \mathbb{T}, d_s = \text{parsePubAttrs}(\alpha_s)$: $\delta \notin d_s$ **then return** 0
      return 1
  return 0

Algorithm 3: The malleability predicate M keeps list of generated addresses and transactions, $\mathbb{L}$ and $\mathbb{T}$, and verifies for rightfully generated addresses.

The early predicate M is used as a parameter in the next functionality. Its role is to specify the level of address malleability we consider in the functionality.

Now, we present the adapted functionality $\mathcal{F}_{\text{HC}}^{M}$.

[6] [14] proposed various predicative parameters for different malleability degrees. We will consider here in this work the *"sink" malleable* predicate, although we drop the term "*sink malleable*" for readability.

Functionality $\mathcal{F}_{\text{HC}}^{M}$

Initialization: On receiving the message $(\text{INIT}, sid, \mathtt{st}_0, \mathtt{mpk}_{cold}^{pay}, \mathsf{id})$ from $P \in \mathbb{P}$, forward it to $\mathcal{S}$ and wait for (INITOK, sid). Then initialize the empty lists L_P of addresses and attribute lists and K_P of staking keys, return (INITOK, sid).

Address Generation: On receiving $(\text{GENERATEADDRESS}, sid, aux, wallet)$ from $P \in \mathbb{P}$, forward it to $\mathcal{S}$. On $(\text{ADDRESS}, sid, \alpha, l_\alpha)$ from $\mathcal{S}$, parse l_α as $(\delta_1, \ldots, \delta_g)$ and $\forall P' \in \mathbb{P}$ check if $\forall(\alpha', (\delta'_1, \ldots, \delta'_g)) \in L_{P'}$ it holds that $\alpha \neq \alpha'$, $\delta'_2 \neq \delta_2$, and $\forall j \in [i, \ldots, g] : \delta'_j \neq \delta_j$, *i.e.*, the address, recovery tag, and private attributes are unique. If so, then, if the following checks hold,

– if $aux =$ ("base"), check that $\forall(\alpha', (\delta'_1, \ldots, \delta'_g)) \in L_P : \delta'_1 \neq \delta_1$
– else if $aux =$ ("pointer", pk^{stk}), check that $\delta_1 = \mathsf{pk}^{stk}$
– else if $aux =$ ("exile"), check that $\delta_1 = \perp$

or P is corrupted, then insert (α, l_α) to L_P and return $(\text{ADDRESS}, sid, \alpha)$ to P. If $aux =$ ("base") also insert δ_1 to K_P and return $(\text{STAKINGKEY}, sid, \delta_1)$ to P.

Wallet Recovery: Upon receiving the message $(\text{RECOVERWALLET}, sid, wallet, i)$ from $P \in \mathbb{P}$, for the first i elements in L_P return $(\text{TAG}, sid, \delta_2)$.

Address Recovery: Upon receiving the message $(\text{RECOVERADDR}, sid, wallet, \alpha, i)$ from P, if (α, l) is one of the first i elements of L_P or $M(L_P, \text{"recover"}, \alpha) = 1$, return $(\text{RECOVEREDADDR}, sid, \alpha)$.

Issue Hot Transaction: Upon receiving $(\text{PAY}, sid, tx = (\Theta, \alpha_\mathsf{s}, \alpha_\mathsf{r}, m))$ from $P \in \mathbb{P}$, if $\exists l_\alpha : (\alpha_\mathsf{s}, l_\alpha) \in L_P$ forward it to $\mathcal{S}$. Upon receiving $(\text{TRANSACTION}, sid, tx, \sigma)$ from $\mathcal{S}$, check if $\forall(tx', \sigma', b') \in \mathcal{T} : \sigma' \neq \sigma$, $(tx, \sigma, 0) \notin \mathcal{T}$, and $M(L_P, \text{"issue"}, \alpha_r) = 1$. If all checks hold, then insert $(tx, \sigma, 1)$ to $\mathcal{T}$, return $(\text{TRANSACTION}, sid, tx, \sigma)$.

Verify Transaction: Upon receiving $(\text{VERIFYPAY}, sid, tx, \sigma)$ from $P \in \mathbb{P}$, with $tx = (\Theta, \alpha_\mathsf{s}, \alpha_\mathsf{r}, m)$ for a metadata string m, forward it to $\mathcal{S}$ and wait for the message $(\text{VERIFIEDPAY}, sid, tx, \sigma, \phi)$. Then

– if $M(L_P, \text{"verify"}, \alpha_s) = 0$, set $f = 0$
– else if $(tx, \sigma, 1) \in \mathcal{T}$, set $f = 1$
– else, if P is not corrupted and $(tx, \sigma, 1) \notin \mathcal{T}$, set $f = 0$ and insert $(tx, \sigma, 0)$ to $\mathcal{T}$
– else, if $(\Theta, \alpha_s, \alpha_r, m, \sigma, b) \in \mathcal{T}$, set $f = b$
– else, set $f = \phi$

Finally, send $(\text{VERIFIEDPAY}, sid, tx, \sigma, f)$ to P.

Issue Staking: Upon receiving the message $(\text{STAKE}, sid, stx, wallet)$ from P, such that $stx = (\mathsf{pk}^{stk}, m)$ for a metadata string m, forward the message to $\mathcal{S}$. On receiving $(\text{STAKED}, sid, stx, \sigma)$ from $\mathcal{S}$, if $\forall(stx', \sigma', b') \in S : \sigma' \neq \sigma$, $(stx, \sigma, 0) \notin S$, and $\mathsf{pk}^{stk} \in K_P$, add $(stx, \sigma, 1)$ to S and return the message $(\text{STAKED}, sid, stx, \sigma)$ to P.

Verify Staking: Upon $(\text{VERIFYSTAKE}, sid, stx, \sigma)$ from $P \in \mathbb{P}$, forward it to $\mathcal{S}$ and wait for the message $(\text{VERIFIEDSTAKE}, sid, stx, \sigma, \phi)$, with $stx = (\mathsf{pk}^{stk}, m)$. Then find P_s such that $\mathsf{pk}^{stk} \in K_{P_\mathsf{s}}$. Then

– if $(stx, \sigma, 1) \in S$, set $f = 1$
– else if P_s is not corrupted and $(stx, \sigma, 1) \notin S$, set $f = 0$ and insert $(stx, \sigma, 0)$ to S
– else if exists an entry $(stx, \sigma, f') \in S$, set $f = f'$
– else set $f = \phi$ and insert (stx, σ, ϕ) to S

Finally, return $(\text{VERIFIEDSTAKE}, sid, stx, \sigma, f)$ to P.

We are finally ready to present our main theorem.

5.2 Main Theorem

From now we present our main security argument, which is based on the UC Framework, and in a nutshell tells us that π_{HC} UC realizes the hot and cold functionality $\mathcal{F}_{\text{HC}}^{M}$.

Theorem 1. *Let the protocol π_{HC} be parameterized by a Stateful Hot/Cold Wallet Σ_{HC} (Definition 6) and the* HKeyGen*,* GenAddr *(Algorithms 2 and 1), and* RTagGen *Functions. Then π_{HC} securely realizes the ideal functionality $\mathcal{F}_{\text{HC}}^{M}$ if, and only if, Σ_{HC} is* EUF-CMA*,* GenAddr *is collision resistant and attribute non-malleable (Definitions 3 and 5),* RTagGen *is collision resistant (Definition 2), and* HKeyGen *is hierarchical for Σ_{HC} (Definition 4).*

Proof-sketch. The proof is analogous to the one in [14]. The main idea is that $\mathcal{F}_{\text{HC}}$ manages two wallets, *i.e.*, two sets of generated addresses, instead of a single set as in [14]. More concretely, note that, in [14], the main argument relies on a core wallet functionality $\mathcal{F}_{\text{Wallet}}$ which specifies the security properties of the PoS Wallet.

The design of $\mathcal{F}_{\text{HC}}$ allows it to deal with two wallets, cold and hot, as if capturing two instances of $\mathcal{F}_{\text{Wallet}}$. In order to see that, note that $\mathcal{F}_{\text{HC}}$ separates the hot interface when issuing (hot payment) transactions, in the *Issue Hot Transaction* Interface, whereas specifying among the parameters of the remaining interfaces the parameter *wallet* to choose between hot and cold. That is, $\mathcal{F}_{\text{HC}}$ handles, simultaneously, two separated wallet interfaces for cold and hot wallets.

Crucially, we remark that there is no interface for issuing cold (payment) transactions, since the wallet cannot sign transactions for cold addresses as it does not have knowledge of the cold (payment) secret key. The absence of interface for issuing cold payment transactions is also highlighted in the π_{HC} protocol which does not have the respective interface.

Finally, we conclude that the security proof for π_{HC} can be accomplished by constructing a simulator $\mathcal{S}_{HC}$ which simulates the execution of the protocol π_{HC} to $\mathcal{A}$ by running internally two copies of the simulator from [14], one for each wallet, cold and hot. Therefore we conclude that the environment has at most negligible probability in distinguishing the ideal execution, between with $\mathcal{F}_{\text{HC}}^{M}$ and $\mathcal{S}_{HC}$, and the real execution, between π_{HC} and $\mathcal{A}$, thereby giving the proof. □

5.3 Addressing Cold/Hot Limitations

Our proof directly shows that it is possible to fulfill all the items from Section 3. In particular, three items, namely 1) **Composable security**, 2) **Cold stake redelegation** and 3) **Cold stake delegation** which seemed more challenging. To that purpose, our proposed protocol π_{HC} crucially exploits the design of the delegation framework: keys for staking and payment. The address issuing and delegation certificate signing require only the cold wallet staking secret key, which can be managed by the hot wallet.

Remark 5. Our protocol is secure in the setting for the level of malleability we have focused, Algorithms 1 and 3. However, we noticed that other more restrictive malleability levels, in the sense of [14], require access to the cold wallet key for payment, *i.e.*, $\mathsf{sk}_{c,id}^{pay}$. For that goal, our protocol can be extended by using hardware aid in the spirit of [10].

6 Final Remarks

The starting point of this work was to identify limitations of the existing delegation framework with respect to hot/cold wallet design. We introduced the first Hold/Cold DPoS wallet protocol π_{HC} and the respective security definition, *i.e.*, Functionality $\mathcal{F}_{\mathrm{HC}}$. We showed that π_{HC} is secure under this security definition in the UC Framework.

Our protocol addresses the limitations we have found, however it relies on less restrictive malleability level (as in [14]). For more restrictive levels, note that our protocol supports hardware based solutions in the spirit of [10]. Our protocol was shown to allow **Cold stake delegation** and **Rewards payment**, in addition to **Composable security** and the trivially a achievable **Hot wallet funds** criteria from Section 3.

Even more crucially, our protocol is shown to perform **Cold stake redelegation** even with a (permanently) off-line wallet, *i.e.*, the cold wallet. It is worth to highlight that our technique exploits the double key design of [14]: a key pair for staking alone and the other for payment transactions for each wallet, with a total of four key pairs.

We leave, for future work, extending this study to the framework which models more closely hardware solutions as introduced by [1], and to more restrictive levels of address malleability, in the sense of [14], with the aid of hardware aid to access the cold wallet, like in [10].

References

1. Myrto Arapinis, Andriana Gkaniatsou, Dimitris Karakostas, and Aggelos Kiayias. A formal treatment of hardware wallets. In Ian Goldberg and Tyler Moore, editors, *FC 2019*, volume 11598 of *LNCS*, pages 426–445. Springer, Cham, February 2019.
2. Jean-Philippe Aumasson and Omer Shlomovits. Attacking threshold wallets. Cryptology ePrint Archive, Report 2020/1052, 2020.
3. Ran Canetti. Universally composable signatures, certification and authentication. Cryptology ePrint Archive, Report 2003/239, 2003.
4. Cardano. Cardano explorer, 2024. `https://https://cexplorer.io/`.
5. Jing Chen and Silvio Micali. Algorand: A secure and efficient distributed ledger. *Theoretical Computer Science*, 777:155–183, 2019.
6. Poulami Das, Andreas Erwig, Sebastian Faust, Julian Loss, and Siavash Riahi. The exact security of BIP32 wallets. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 1020–1042. ACM Press, November 2021.
7. Poulami Das, Andreas Erwig, Sebastian Faust, Julian Loss, and Siavash Riahi. BIP32-compatible threshold wallets. Cryptology ePrint Archive, Report 2023/312, 2023.
8. Poulami Das, Sebastian Faust, and Julian Loss. A formal treatment of deterministic wallets. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 651–668. ACM Press, November 2019.

9. Bernardo David, Peter Gazi, Aggelos Kiayias, and Alexander Russell. Ouroboros praos: An adaptively-secure, semi-synchronous proof-of-stake blockchain. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 66–98. Springer, Cham, April / May 2018.
10. Rafael Baiao Dowsley, Mylene CQ Farias, Mario Larangeira, Anderson CA Nascimento, and Jot Virdee. A spendable cold wallet from qr video. In *International Conference on Security and Cryptography 2022*, pages 283–290. Scitepress, 2022.
11. Shafi Goldwasser, Silvio Micali, and Ronald L. Rivest. A "paradoxical" solution to the signature problem (abstract) (impromptu talk). In G. R. Blakley and David Chaum, editors, *CRYPTO'84*, volume 196 of *LNCS*, page 467. Springer, Berlin, Heidelberg, August 1984.
12. Jens Groth and Victor Shoup. On the security of ECDSA with additive key derivation and presignatures. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 365–396. Springer, Cham, May / June 2022.
13. Dominik Hartmann and Eike Kiltz. Limits in the provable security of ECDSA signatures. In Guy N. Rothblum and Hoeteck Wee, editors, *TCC 2023, Part IV*, volume 14372 of *LNCS*, pages 279–309. Springer, Cham, November / December 2023.
14. Dimitris Karakostas, Aggelos Kiayias, and Mario Larangeira. Account management in proof of stake ledgers. In Clemente Galdi and Vladimir Kolesnikov, editors, *SCN 20*, volume 12238 of *LNCS*, pages 3–23. Springer, Cham, September 2020.
15. Dimitris Karakostas, Aggelos Kiayias, and Mario Larangeira. Conclave: A collective stake pool protocol. In Elisa Bertino, Haya Shulman, and Michael Waidner, editors, *ESORICS 2021, Part I*, volume 12972 of *LNCS*, pages 370–389. Springer, Cham, October 2021.
16. Yashvanth Kondi, Bernardo Magri, Claudio Orlandi, and Omer Shlomovits. Refresh when you wake up: Proactive threshold wallets with offline devices. In *2021 IEEE Symposium on Security and Privacy*, pages 608–625. IEEE Computer Society Press, May 2021.
17. Cardano Settlement Layer. Addresses in Cardano SL . `https://github.com/input-output-hk/cardano-docs/blob/master/docs/cardano/addresses.md`, 2016. [Online; accessed 20-September-2023].
18. Li Li. Mitigating challenges in ethereum's proof-of-stake consensus: Evaluating the impact of eigenlayer and lido, 2024.
19. Gregory Maxwell et al. Deterministic wallets, 2014.
20. Silvio Micali, Michael O. Rabin, and Salil P. Vadhan. Verifiable random functions. In *40th FOCS*, pages 120–130. IEEE Computer Society Press, October 1999.
21. Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. 2008.